Practice Set 4 — Aggregate Functions

Q1. Total number of products, cheapest price, and most expensive price in one query.

```
130 •  SELECT COUNT(*) AS total_products,
131           MIN(price) AS cheapest_price,
132           MAX(price) AS most_expensive_price
133    FROM products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_products	cheapest_price	most_expensive_price
9	25	71100

27 17:09:21 SELECT COUNT(*) AS total_products, MIN(price) AS cheapest_price, MA... 1 row(s) returned

Q2. Total stock of all products, and total stock of Electronics only.

```
136 •  SELECT SUM(stock) AS total_stock,
137           SUM(
138               CASE
139                   WHEN category = 'Electronics' THEN stock
140                   ELSE 0
141               END
142           ) AS electronics_stock
143    FROM products;
144
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_stock	electronics_stock
823	43

28 17:18:57 SELECT SUM(stock) AS total_stock, SUM(CASE WHEN cat... 1 row(s) returned

Q3. Average price per category (GROUP BY).

```
146 •  SELECT category,
147           AVG(price) AS average_price
148    FROM products
149    GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price
Grocery	775.0000
Electronics	49200.0000
Beauty	425.0000
NULL	NULL

29 17:19:58 SELECT category, AVG(price) AS average_price FROM products GROUP BY ... 4 row(s) returned

Q4. Count products per category, but show only categories with at least 2 products. WHERE or HAVING? Why?

```
151 •  SELECT category,
152           COUNT(*) AS total_products
153    FROM products
154    GROUP BY category
155    HAVING COUNT(*) >= 2;
156
157
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_products
Grocery	3
Electronics	3
Beauty	2

30 17:23:11 SELECT category, COUNT(*) AS total_products FROM products GROUP BY c... 3 row(s) returned

Q5. Average price per category, counting only products cheaper than 60000. Which filter goes in WHERE?

```
157 •  SELECT category,
158            AVG(price) AS average_price
159    FROM products
160    WHERE price < 60000
161    GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price
Grocery	775.0000
Electronics	38250.0000
Beauty	425.0000

31 17:24:51 SELECT category, AVG(price) AS average_price FROM products WHERE pric... 3 row(s) returned

Q6. Categories where the average price is above 1000, sorted biggest first. Use the full general order.

```
163 •  SELECT category,
164            AVG(price) AS average_price
165    FROM products
166    GROUP BY category
167    HAVING AVG(price) > 1000
168    ORDER BY average_price DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price
Electronics	49200.0000

32 17:26:00 SELECT category, AVG(price) AS average_price FROM products GROUP BY ... 1 row(s) returned

Q7. Library: total fine per member (JOIN + GROUP BY). Members with no loans must show 0. Which JOIN?

```sql
USE library;

SELECT members.member_id,
       members.mname,
       COALESCE(SUM(loans.fine), 0) AS total_fine
FROM members
LEFT JOIN loans
ON members.member_id = loans.member_id
GROUP BY members.member_id, members.mname;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	mname	total_fine
1	Ram	170
2	Sita	110
3	Hari	40
4	Gita	170
5	Nabin	0

```
34  17:27:16  USE library                                                    0 row(s) affected
35  17:27:21  SELECT members.member_id,  members.mname,  COALESCE(SUM(loans.fi...  5 row(s) returned
```

Q8. Library: members whose total fine is above 100 (HAVING).

```sql
SELECT members.member_id,
       members.mname,
       SUM(loans.fine) AS total_fine
FROM members
INNER JOIN loans
ON members.member_id = loans.member_id
GROUP BY members.member_id, members.mname
HAVING SUM(loans.fine) > 100;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	mname	total_fine
1	Ram	170
2	Sita	110
4	Gita	170

```
36  17:28:57  SELECT members.member_id,  members.mname,  SUM(loans.fine) AS total_...  3 row(s) returned
```

Q9. COUNT(*) vs COUNT(fine): make one fine NULL and show the difference.

```
190 •   UPDATE loans
191     SET fine = NULL
192     WHERE loan_id = 101;
193
194 •   SELECT COUNT(*) AS total_rows,
195            COUNT(fine) AS non_null_fines
196     FROM loans;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_rows	non_null_fines
8	7

37	17:30:10	UPDATE loans SET fine = NULL WHERE loan_id = 101	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
38	17:30:16	SELECT COUNT(*) AS total_rows, COUNT(fine) AS non_null_fines FROM loan...	1 row(s) returned

Q10. In one sentence: why can HAVING see COUNT(*) but WHERE cannot?
Where ilters rows before grouping,while having filters groups after aggregate functions such as count () have been calculated.